

Analiza Kodu ML

1. Plik: ml_models.py (Fabryka Modeli)

Ten plik służy jako magazyn. Przechowujemy w nim definicje wszystkich algorytmów.

```
from sklearn.ensemble import GradientBoostingClassifier, GradientBoostingRegressor,
RandomForestClassifier, RandomForestRegressor
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
```

Co to robi? Importujemy gotowe "silniki" matematyczne z najpopularniejszej biblioteki ML w Pythonie (scikit-learn). Dzięki temu nie musimy pisać algorytmu Random Forest od zera.

```
@dataclass(frozen=True)
class ModelSpec:
    method_name: str
    family: str
    owner: str
    problem_type: str
    factory: EstimatorFactory
    default_params: dict
```

Co to robi? To jest "szablon" (struktura danych). Mówi nam, że każdy model w naszym systemie musi mieć nazwę, autora (np. person2), typ (klasyfikacja/regresja) oraz funkcję budującą (factory) i słownik domyślnych parametrów (default_params). Słowo @dataclass sprawia, że Python sam tworzy dla nas ładny obiekt bez pisania zbędnego kodu, a frozen=True oznacza, że nikt nie może w trakcie działania programu nagle zmienić tych definicji (ochrona przed błędami).

```
def _random_forest_classifier(**kwargs):
    return RandomForestClassifier(**kwargs)
```

Co to robi? To funkcja-fabryka. Kiedy jej użyjemy, zwraca nam nowy, świeży obiekt algorytmu Random Forest. **kwargs oznacza, że może przyjąć dowolną liczbę parametrów ukrytych (np. głębokość drzewa).

```

MODEL_REGISTRY: dict[str, ModelSpec] = {
    "random_forest_classification": ModelSpec(
        method_name="random_forest",
        family="tree_ensemble",
        owner="person2",
        problem_type="classification",
        factory=_random_forest_classifier,
        default_params={"criterion": "gini", "n_jobs": -1},
    ),
    # ... inne modele ...
}

```

Co to robi? Tworzymy ogromny Słownik (dict). To serce tego pliku. Kiedy jakkolwiek inny skrypt w projekcie powie "daj mi random_forest_classification", program zagłąda tutaj, bierze ten obiekt i ma od razu podane wszystkie właściwości modelu oraz sposób jego stworzenia (factory). n_jobs=-1 oznacza "użyj wszystkich rdzeni procesora, żeby liczyło się szybciej".

2. Plik: ml_config.py (Konfigurator Eksperymentów)

Ten plik to po prostu lista zadań (testów), które zlecamy naszemu komputerowi.

```

@dataclass(frozen=True)
class ExperimentDefinition:
    experiment_id: str
    task_name: str
    model_key: str
    owner: str
    tuned_param: str
    values: list
    fixed_params: dict
    primary_metric: str
    notes: str

```

Co to robi? Ponownie tworzymy "szablon". Tym razem opisuje on pojedynczy test (np. testowanie głębokości drzewa). tuned_param to nazwa parametru, który będziemy zmieniać (np. max_depth), a values to lista wartości, przez które przejdziemy (np. [5, 10, 15]). fixed_params to reszta ustawień, których nie ruszamy.

```

UM_EXPERIMENTS: list[ExperimentDefinition] = [
    ExperimentDefinition(
        experiment_id="classification_divorce_random_forest_n_estimators",
        task_name="classification_divorce",
        model_key="random_forest_classification",
        owner="person2",
        tuned_param="n_estimators",
        values=[50, 100, 200, 300],
        fixed_params={"max_depth": 10},
        primary_metric="balanced_accuracy",
        notes="",
    ),
    # ... kilkanaście innych eksperymentów ...
]

```

Co to robi? To jest nasza główna tablica ogłoszeń. Dodaliśmy tutaj obiekt mówiący: *"Dla klasyfikacji rozwodu, weź model Random Forest. Będziemy testować parametr n_estimators po kolei dla wartości 50, 100, 200, 300. Pozostałe ustawienia (jak głębokość) zablokuj na 10".*

3. Plik: ml_experiments.py (Główny Silnik)

To najbardziej skomplikowany plik. To on fizycznie łączy plik z danymi z wybranym modelem.

```

def load_processed_dataset(task_name: str) -> tuple[np.ndarray, np.ndarray,
np.ndarray, np.ndarray, dict]:
    npz_path = PROCESSED_DATA_DIR / f"{task_name}.npz"
    metadata_path = PROCESSED_DATA_DIR / f"{task_name}_metadata.json"

```

Co to robi? Ta funkcja otwiera wygenerowane przez nas przygotowane pliki matematyczne z danymi. Plik .npz to spakowane macierze (tabele z liczbami z Excela).

```

data = np.load(npz_path)
return data["X_train"], data["X_test"], data["y_train"], data["y_test"], metadata

```

Co to robi? Wypakowuje z pliku .npz cztery główne obiekty: X_train (dane demograficzne do nauki), y_train (poprawne odpowiedzi z ankiety dla tych ludzi), X_test (dane nowych pacjentów do egzaminu) oraz y_test (odpowiedzi do egzaminu, żeby potem móc ocenić model).

```
class UMLExperimentRunner:
    def __init__(self, seeds: list[int] | None = None, ...):
        self.seeds = seeds or UM_RANDOM_SEEDS
```

Co to robi? Startujemy klasę, która będzie zarządzać biegami modelu. Inicjalizuje ona w sobie seeds (czyli np. wartości [42, 43, 44]). Ziarna losowości są nam potrzebne, by móc odpalić model wielokrotnie i wyciągnąć średnią z wyniku. Chroni to badacza przed fuksem statystycznym.

```
def run_definition(self, definition: ExperimentDefinition) -> pd.DataFrame:
    X_train, X_test, y_train, y_test, metadata =
load_processed_dataset(definition.task_name)
    model_spec = MODEL_REGISTRY[definition.model_key]
```

Co to robi? Serce programu. Bierze nasze zapytanie z ml_config.py. Ładuje odpowiednie dane, a następnie "dzwoni" do naszej fabryki (MODEL_REGISTRY) podając jej klucz (np. "knn_regression").

```
for value in definition.values:
    for repeat_idx, seed in enumerate(self.seeds, start=1):
```

Co to robi? Podwójna pętla. Pierwsza bierze pierwszą wartość do testu (np. max_depth = 5). Druga pętla zarządza powtórzeniami (odpala ten sam test 3 razy na innych ziarnach).

```
params = dict(model_spec.default_params)
params.update(definition.fixed_params)
params[definition.tuned_param] = value
```

Co to robi? Zlepia wszystkie parametry w jeden pakunek. Bierze domyślne ustawienia, nadpisuje te stałe (np. n_estimators=100), a na koniec wgrywa ten jeden konkretny parametr, który akurat testujemy (max_depth = 5).

```
estimator = model_spec.factory(**params)
estimator.fit(X_train, y_train.reshape(-1))
```

Co to robi? (1) Uruchamia funkcję factory, co fizycznie tworzy pusty obiekt modelu matematycznego. (2) **Najważniejsza linijka w Machine Learningu:** fit(). Zmusza model do przeanalizowania danych treningowych i nauczania się, skąd się biorą rozwody.

```
y_train_pred = estimator.predict(X_train)
y_test_pred = estimator.predict(X_test)
```

Co to robi? Każe modelowi zgadnąć odpowiedź. predict(X_train) to prośba o rozwiązanie egzaminu ze ściągą (sprawdzamy na danych, które widział). predict(X_test) to prawdziwy egzamin (zgadywanie z obcych, nowych danych).

```
if problem_type == "classification":
    train_metrics = evaluate_classification(y_train, y_train_pred)
    test_metrics = evaluate_classification(y_test, y_test_pred)
```

Co to robi? Wysyła przewidywania naszego modelu do funkcji oceniającej w ml_metrics.py.

4. Plik: ml_metrics.py (Komisja Egzaminacyjna)

Ten plik pełni rolę bezstronnego weryfikatora. Sprawdza, czy model poprawnie zgadł wyniki na egzaminie.

```
def evaluate_classification(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    y_true = np.asarray(y_true).reshape(-1)
    y_pred = np.asarray(y_pred).reshape(-1)
```

Co to robi? Przyjmuje dwie listy. y_true to odpowiedzi z prawdziwego "klucza egzaminatora" (rzeczywiste rozwody). y_pred to odpowiedzi strzelone przez model na teście. Następnie prostuje te listy (.reshape(-1)), aby mieć pewność, że to płaskie ciągi zer i jedynek.

```
return {
    "accuracy": float(accuracy_score(y_true, y_pred)),
```

```

    "balanced_accuracy": float(balanced_accuracy_score(y_true, y_pred)),
    "f1": float(f1_score(y_true, y_pred, zero_division=0)),
}

```

Co to robi? Oblicza i zwraca paczkę ostatecznych ocen.

- **Accuracy:** Jaki procent wszystkich odpowiedzi był poprawny (często kłamie).
- **Balanced Accuracy:** Najważniejsza metryka dla nas! Bierze pod uwagę to, że osób rozwiedzionych jest mało (tylko 10%) i ocenia skuteczność algorytmu SPRAWIEDLIWIE na obu podgrupach (nierozwiedzonych i rozwiedzionych osobno, a na koniec wyciąga z nich średnią).
- `zero_division=0` zapobiega "wybuchowi" programu z powodu dzielenia przez zero, gdyby model w ogóle nie znalazł rozvodu.

```

def evaluate_regression(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    mse = float(mean_squared_error(y_true, y_pred))
    return {
        "rmse": float(np.sqrt(mse)),
        "mae": float(mean_absolute_error(y_true, y_pred)),
        "r2": float(r2_score(y_true, y_pred)),
    }

```

Co to robi? Analogiczna komisja egzaminacyjna dla zadania drugiego (Staż Matżeński, czyli zmienna ciągła, a nie 0/1).

- **MSE:** błąd średniokwadratowy. Podnosi błąd do potęgi, surowo karząc model za duże pomyłki (np. pomylenie stażu o 10 lat to kara aż 100 punktów).
- **RMSE:** Zwykły pierwiastek z MSE (dzięki temu mamy czytelny błąd podany w latach, a nie "kwadratowych latach").
- **MAE:** Bezwzględny średni błąd – o ile lat średnio myli się model patrząc z lotu ptaka.

5. Plik: `run_feature_importance.py` (Skrypt Ważności Cech)

Ten specjalny plik z informacją "dlaczego podjęta taką decyzję".

```

def run_feature_importance():
    X_train, X_test, y_train, y_test, metadata =

```

```
load_processed_dataset("classification_divorce")
```

Co to robi? Tak jak w przypadku treningu normalnego, ładujemy pakiet tabelaryczny do pamięci operacyjnej RAM.

```
feature_names = metadata.get("feature_names_after_preprocessing", ...)
```

Co to robi? Nasz program na etapie przygotowywania danych zamienił np. kolumnę Religia w kilka kolumn Religia_Katolik, Religia_Inna. Ta linijka wyciąga słownik ("mapę") wszystkich 39 wygenerowanych w ten sposób nazw.

```
rf = RandomForestClassifier(n_estimators=200, random_state=42,  
class_weight='balanced', n_jobs=-1)  
rf.fit(X_train, y_train)
```

Co to robi? Inicjujemy pojedynczy model Lasu Losowego. Kluczowym parametrem tu wpisanym jest `class_weight='balanced'`. Oznacza to: *"zwróć silną uwagę na osoby rozwiedzione, mimo że jest ich 10 razy mniej w pliku"*. Następnie wywołujemy słynne `.fit()`, żeby zbudował drzewa decyzyjne.

```
importances = rf.feature_importances_
```

Co to robi? Magia biblioteki scikit-learn. Las Losowy w trakcie trenowania sam prowadzi księgowość. Kiedy zauważy, że użycie np. Stażu małżeńskiego drastycznie ułatwiło mu wyłapanie rozwodu, zapisuje temu stażowi wysoki punkt procentowy. Ta linijka pobiera ostateczne statystyki tej "wewnętrznej matematyki" prosto ze środka lasu do zmiennej.

```
clean_names = [name.replace('num__', '').replace('cat__', '') for name in  
feature_names]  
indices = np.argsort(importances)[::-1]  
sorted_names = [clean_names[i] for i in indices]
```

Co to robi? Sprzątanie w nazwach. System Pipeline dodał z przodu słowa `num__` (dla liczb) i `cat__` (dla słów). Ta jednolijkowa pętla (`replace`) wycina te brzydkie przedrostki, a `np.argsort()[::-1]` ustawia cechy posortowane od najważniejszej do najśłabszej.

```
plt.barh(top_names[::-1], topimps[::-1], color='darkred')  
plt.title("Top najważniejszych czynników decydujących o rozwodzie")
```

```
plot_path = PLOTS_DIR / "classification_divorce_feature_importance.png"  
plt.savefig(plot_path, dpi=300)
```

Co to robi? Kawałek kodu odpowiedzialny wyłącznie za oprawę wizualną. Tworzy tzw. wykres słupkowy poziomy (barh - Horizontal Bar). Ustawia kolor na ciemnoczerwony i zapisuje pod postacią ślicznego, ważącego kilkaset kilobajtów (bo ma gęstość dpi=300 px) obrazka na dysk twardy, którego ostatecznie użyliście w Raporcie.